UniStay – Hostel Room Allocation and Swap Management System

Project submitted to the

School of Engineering & Technology, BML Munjal University

for the partial fulfilment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Technology

Submitted by: Group 6

Ananya Upadhyay(240378)
Annu(240385)
Abhinav Sharma(240362)
Aditya Narayan Singh(240371)

BML MUNJAL UNIVERSITY
FROM HERE TO THE WORLD

Under the Guidance of
Dr. Nishtha Phutela

BML Munjal University
Kapriwas, Haryana – 122413

6 May, 2026

Certificate

Date: 5 May 2026

This is to certify that the work present in this Project entitled "**UniStay – Hostel Room Allocation and Swap Management System**" has been carried out by **Ananya Upadhyay, Annu, Abhinav Sharma, and Aditya Narayan Singh** under my supervision. The work is genuine, original, and suitable for submission to BML Munjal University for the award of Bachelor of Technology in School of Engineering and Technology.

Supervisor

Dr. Nishtha Phutela

BML Munjal University

Table of Contents

Sl. No.	Topic	Page No.
1	Certificate	2
2	Table of Contents	3
3	Abstract	4
4	Abbreviations	5
5	List of Tables	6
6	List of Figures	6
7	Introduction	7
8	Problem Statement	8
9	Methodology	9
10	Discussion / Results	14
11	Concluding Remarks	17
12	Societal Relevance	17
13	Future Scope	18
14	Team Details and Contribution	18
15	References	20

Abstract

UniStay is a web-based Hostel Room Allocation and Swap Management System designed to overcome the inefficiencies of traditional manual hostel administration in university campuses. In conventional systems, students are required to physically visit hostel offices, fill out paper forms, and wait for room confirmations, making the process slow, error-prone, and often unfair. UniStay replaces this outdated workflow with a centralized digital platform that enables students to register, view available rooms, and submit allocation requests through a web interface. The system maintains a real-time database of room availability and enforces constraints such as gender compatibility, room capacity, and booking status.

The system is developed using MySQL as the relational database, Node.js as the backend server, and HTML/CSS for the frontend interface. The database is designed using normalized tables such as Students, Hostels, Rooms, Allocations, Swap_Requests, and Admins, ensuring data integrity through foreign key relationships. Advanced features like triggers, stored procedures, and views are used to enforce business rules, prevent overbooking, and automate occupancy management. Additionally, a structured room swap mechanism allows students to request exchanges, which are validated through peer consent and administrative approval. The system provides a scalable, transparent, and efficient solution that reduces administrative workload and improves fairness in hostel allocation.

Abbreviations

Short Form	Full Form
DBMS	Database Management System
ER	Entity-Relationship
SQL	Structured Query Language
UI	User Interface
DDL	Data Definition Language
DML	Data Manipulation Language
DQL	Data Query Language
API	Application Programming Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
Node.js	Node JavaScript Runtime Environment
HTTP	HyperText Transfer Protocol
REST	Representational State Transfer
ACID	Atomicity, Consistency, Isolation, Durability
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
PK	Primary Key
FK	Foreign Key
AI	Artificial Intelligence
IDE	Integrated Development Environment
OS	Operating System

List of Tables

Table 1. Abbreviations and their Full Forms

Table 2. Entity Summary (Entities, Attributes, Primary Keys)

Table 3. Database Schema (Tables, Columns, Constraints)

Table 4. Normalization Summary (Normal Forms Applied)

Table 5. Room Allocation Scenario (Sample Data and Results)

Table 6. Swap Request Scenario (Sample Data and Results)

Table 7. Query Results (Students with Approved Allocations)

Table 8. Team Details and Contributions

List of Figures

Figure 1. ER Diagram – Entity-Relationship diagram of the UniStay database

Figure 2. System Architecture – Three-tier architecture (Frontend, Backend, Database)

Figure 3. Room Allocation Flowchart – Process flow of room allocation algorithm

Figure 4. Swap Request Workflow – Approval process for room exchange

Figure 5. Plagiarism check

Introduction

Managing hostel accommodation in a university environment is a multifaceted administrative challenge that affects thousands of students and staff every academic year. Traditionally, hostel room allocation has been handled manually students submit paper-based applications, wardens review them individually, rooms are assigned by hand, and swap requests are processed through face-to-face meetings and written consent forms. This approach, while simple in concept, is highly inefficient in practice. It leads to long waiting times, misallocation of rooms due to human error, lack of transparency for students, and an overwhelming workload for hostel administrators.

The rapid growth of university enrolments has further strained these manual systems. A single warden may be responsible for hundreds of students, making it practically impossible to track occupancy, handle swap requests, and manage approvals in real time without the aid of technology. Students often face uncertainty about their accommodation status, and disputes over room assignments are common when records are maintained on paper or in disconnected spreadsheets.

UniStay was conceived as a direct solution to these problems. It is a web-based Hostel Room Allocation and Swap Management System that brings the entire hostel administration workflow onto a centralized digital platform. The system serves two categories of users: students and wardens (admins). Students can register on the platform, view a real-time list of available rooms filtered by gender compatibility, submit room allocation requests, request swaps with other students, and track the status of their requests. Wardens can view all pending and approved allocations, approve or reject requests, manage swap approvals, and oversee the overall hostel occupancy.

The system is built around a normalized relational database with six core tables: Admins, Students, Hostels, Rooms, Allocations, and Swap_Requests. The database is powered by MySQL and incorporates advanced features such as stored procedures (for room allocation and swap approval logic), database triggers (to enforce capacity limits and prevent cross-gender swaps), and views (to simplify complex queries for the application layer). The backend is implemented in Node.js, which communicates with the MySQL database using a connection pool, and serves a lightweight HTML/CSS frontend. Together, these components deliver a reliable, scalable, and user-friendly hostel management solution that eliminates paperwork, reduces errors, and improves the experience for both students and administrators.

Problem Statement

The current manual hostel room allocation process at university campuses suffers from several critical shortcomings:

- **Inefficiency:** Room allocation requires physical visits, paper forms, and manual cross-checking of occupancy records, making the process slow and resource-intensive.
- **Lack of real-time information:** Students have no way to view which rooms are available at any given time, leading to confusion and repeated queries to hostel staff.
- **Error-prone manual records:** Maintaining allocation data in registers or spreadsheets is susceptible to human errors, including double allocation of rooms and incorrect capacity tracking.
- **Opaque swap process:** Room swap requests require physical consent from the target student and manual warden approval, with no structured workflow to track the status of a request.
- **Administrative overload:** Wardens spend a disproportionate amount of time on administrative tasks, reducing the time available for student welfare and hostel management.

UniStay directly addresses all of these issues by providing a digital system with automated constraint enforcement, real-time availability tracking, a structured approval workflow, and a clear audit trail for all allocation and swap activities.

Methodology

5.1 Design

ER Diagram

The Entity-Relationship (ER) diagram for UniStay models the core entities and their relationships within the hostel management domain. The following entities were identified during the design phase:

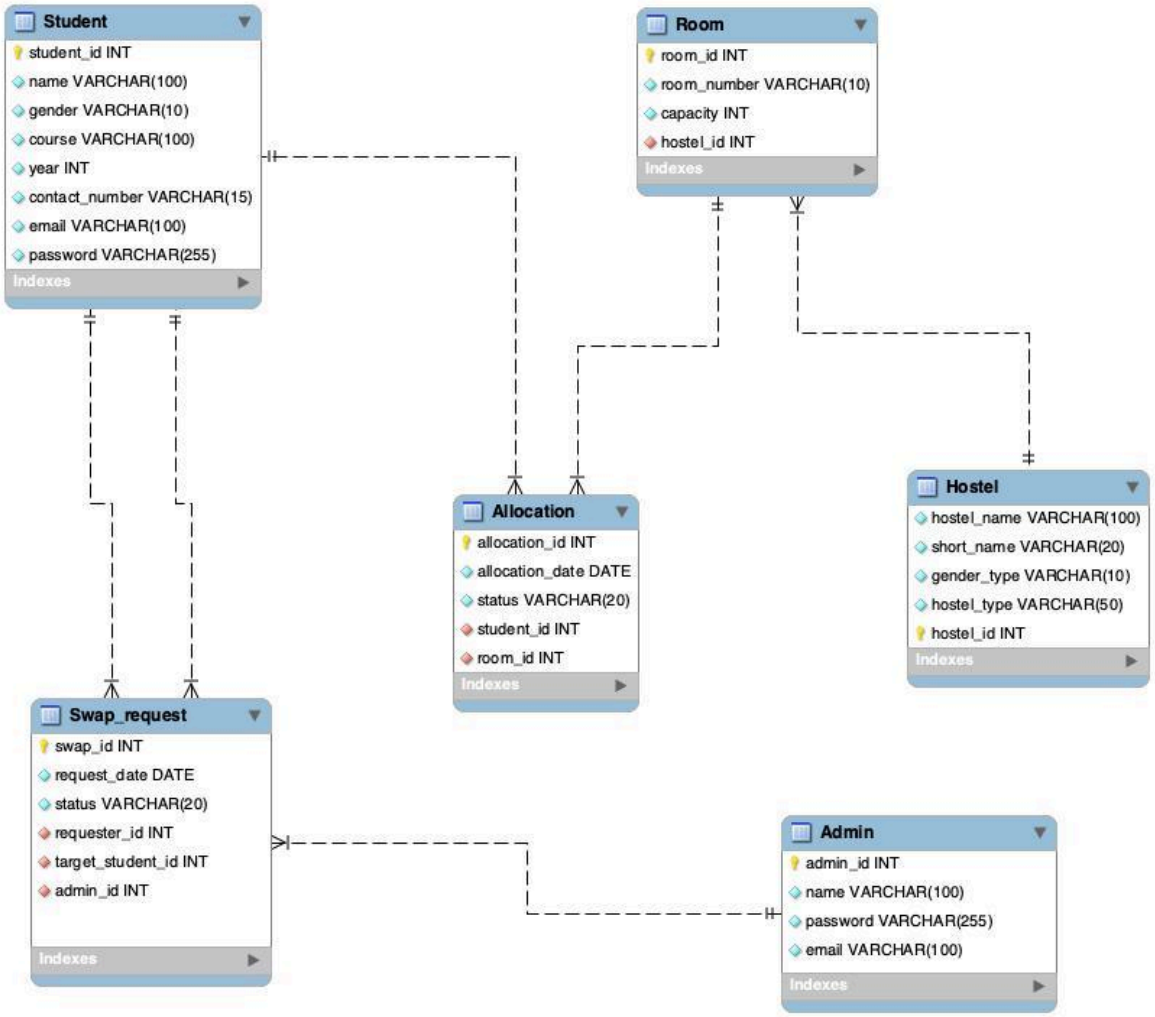

Figure 1. ER Diagram – Entity-Relationship diagram of the UniStay database

- Students: Represents registered students. Attributes include student_id (PK), name, gender, course, year, contact_number, email, and password. Gender is constrained to Male, Female, or Other.
- Admins: Represents hostel wardens and administrative staff. Attributes include admin_id (PK), name, email, and password.
- Hostels: Represents hostel buildings on campus. Attributes include hostel_id (PK), hostel_name, gender_type (Boys, Girls, or Co-ed), and hostel_type (AC or Non-AC).

- Rooms: Represents individual rooms within a hostel. Attributes include room_id (PK), room_number, capacity, and hostel_id (FK referencing Hostels).
- Allocations: Represents the assignment of a student to a room. Attributes include allocation_id (PK), student_id (FK, UNIQUE), room_id (FK), allocation_date, and status (Pending, Approved, or Rejected).
- Swap_Requests: Represents a room swap request between two students. Attributes include swap_id (PK), requester_id (FK), target_student_id (FK), target_consent, status, admin_id (FK, nullable), and request_date.

Key relationships: A Student has at most one Allocation (1:1 via UNIQUE constraint). A Room belongs to one Hostel, and a Hostel has many Rooms (1:N). An Allocation links one Student to one Room (N:1). A Swap_Request links two Students and is optionally processed by one Admin.

Entity	Primary Key	Key Attributes	Relationships
Students	student_id	name, gender, email, course, year	Has one Allocation; Can be Requester or Target in Swap_Requests
Admins	admin_id	name, email	Processes Swap_Requests
Hostels	hostel_id	hostel_name, gender_type, hostel_type	Contains many Rooms
Rooms	room_id	room_number, capacity, hostel_id	Belongs to one Hostel; Referenced by Allocations
Allocations	allocation_id	student_id, room_id, status, allocation_date	Links Student to Room
Swap_Requests	swap_id	requester_id, target_student_id, status, admin_id	Links two Students; Processed by Admin

Table 2: ER Entity Summary

Database Schema

The database schema defines the full structure of all six tables, including column types, constraints, and foreign key relationships. The schema was designed to enforce data integrity at the database level, ensuring that invalid data cannot be inserted even if the application layer fails to validate it.

Table	Column	Type	Constraints
Admins	admin_id	INT	PK, AUTO_INCREMENT

Table	Column	Type	Constraints
Admins	name	VARCHAR(100)	NOT NULL
Admins	email	VARCHAR(150)	UNIQUE, NOT NULL
Admins	password	VARCHAR(255)	NOT NULL
Students	student_id	INT	PK, AUTO_INCREMENT
Students	name	VARCHAR(100)	NOT NULL
Students	gender	VARCHAR(10)	CHECK IN ('Male','Female','Other')
Students	course	VARCHAR(100)	NOT NULL
Students	year	INT	CHECK year BETWEEN 1 AND 5
Students	email	VARCHAR(150)	UNIQUE, NOT NULL
Hostels	hostel_id	INT	PK, AUTO_INCREMENT
Hostels	gender_type	VARCHAR(10)	CHECK IN ('Boys','Girls','Co-ed')
Rooms	room_id	INT	PK, AUTO_INCREMENT
Rooms	capacity	INT	CHECK capacity > 0
Rooms	hostel_id	INT	FK -> Hostels ON DELETE CASCADE
Allocations	student_id	INT	FK -> Students, UNIQUE
Allocations	status	VARCHAR(20)	CHECK IN ('Pending','Approved','Rejected')
Swap_Requests	requester_id	INT	FK -> Students ON DELETE CASCADE
Swap_Requests	admin_id	INT	FK -> Admins ON DELETE SET NULL
Swap_Requests	requester_id != target_student_id	—	CHECK constraint

Table 3: Database Schema Summary

Normalization

The UniStay database was designed in compliance with the Third Normal Form (3NF) to eliminate data redundancy and ensure data consistency:

- **First Normal Form** (1NF): All tables have atomic column values and a defined primary key. There are no repeating groups or multi-valued attributes.

- **Second Normal Form** (2NF): All non-key attributes are fully functionally dependent on the entire primary key. Since all tables use single-column surrogate primary keys (INT AUTO_INCREMENT), partial dependencies are structurally impossible.
- **Third Normal Form** (3NF): There are no transitive dependencies. For example, hostel details (name, gender_type) are stored in the Hostels table and referenced by Rooms through hostel_id, rather than duplicated in each Room row.

Table	1NF	2NF	3NF	Notes
Students	Yes	Yes	Yes	All attributes depend solely on student_id
Admins	Yes	Yes	Yes	All attributes depend solely on admin_id
Hostels	Yes	Yes	Yes	No transitive dependencies
Rooms	Yes	Yes	Yes	hostel_id is FK, not embedded hostel data
Allocations	Yes	Yes	Yes	No repeated or derived attributes
Swap_Requests	Yes	Yes	Yes	All attributes are atomic and non-transitive

Table 4: Normalization Summary

5.2 Implementation

The UniStay system was implemented as a three-tier web application comprising a frontend presentation layer, a backend application layer, and a relational database layer.

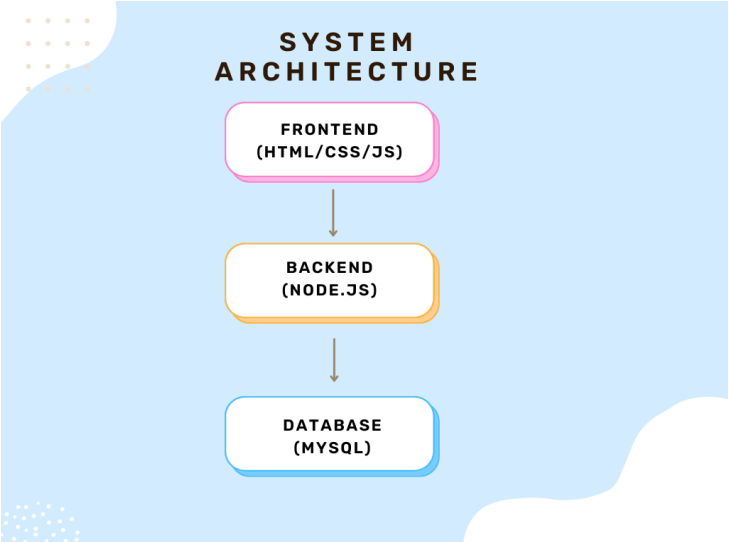

Figure 2. System Architecture – Three-tier architecture (Frontend, Backend, Database)

Database Layer (MySQL)

The database is the core of the system. The schema was defined in schema.sql and sets up all six tables with appropriate constraints. Advanced database objects were created in advanced.sql:

- **Views:** View_Allocated_Students and View_Available_Rooms provide precomputed, always-fresh snapshots of allocation status and room availability. These views are queried by the backend to serve dashboard data without complex JOIN queries in application code.
- **Stored Procedures:** AllocateRoom checks for existing allocations, verifies gender compatibility, and confirms room capacity before inserting a new allocation as 'Pending'. ApproveSwap uses a transaction with row-level locking to atomically swap room_id values between two students, ensuring data consistency even under concurrent access.
- **Triggers:** trg_prevent_over_capacity fires BEFORE INSERT on Allocations to block overbooking. trg_prevent_over_capacity_update fires BEFORE UPDATE to prevent approving an allocation when the room is already full. trg_prevent_cross_gender_swap fires BEFORE INSERT on Swap_Requests to block swaps between students of different genders.
- **Indexes:** Created on Students(email), Allocations(status), Rooms(hostel_id), and Swap_Requests(status) to accelerate the most frequent query patterns.

Backend Layer (Node.js)

The backend is built with Node.js and uses the mysql2 library for database communication. A connection pool (configured in index.js) manages up to 10 simultaneous MySQL connections, which are shared across all route files. The pool's execute() method is used for all queries, providing automatic prepared statement handling and protection against SQL injection.

The backend exposes REST API endpoints for student registration and login, room browsing, allocation requests (which call the AllocateRoom stored procedure), swap request creation, admin approval workflows, and dashboard data (queried from views).

Frontend Layer (HTML/CSS)

The frontend is implemented as static HTML pages styled with CSS. Separate pages are provided for the student login and registration, the student dashboard (showing allocation status and available rooms), the swap request form, the admin login, and the admin dashboard (showing pending allocations and swap requests with approve/reject actions). Forms submit data to the Node.js backend via HTTP POST requests.

5.3 Algorithm

Algorithm 1: Room Allocation Logic

Input: Student ID (p_student_id), Room ID (p_room_id)

Output: New allocation record inserted with status 'Pending', OR an error message if any constraint is violated.

- **Step 1:** Query the Allocations table. If a record exists for p_student_id with status 'Approved' or 'Pending', raise error: 'Student already has a pending or approved allocation.'
- **Step 2:** Retrieve the gender of the student from the Students table. Retrieve the gender_type of the hostel associated with p_room_id by joining Rooms and Hostels.
- **Step 3:** If gender_type is not 'Co-ed', check compatibility: Male students can only be allocated to 'Boys' hostels; Female students can only be allocated to 'Girls' hostels. If mismatch is detected, raise error: 'Gender mismatch: You cannot book a room in this hostel.'
- **Step 4:** Retrieve the capacity of the room (p_room_id). Count the number of existing 'Approved' allocations for that room.
- **Step 5:** If current occupancy >= capacity, raise error: 'Room is fully occupied. Allocation failed.'
- **Step 6:** All checks passed. Insert a new record into Allocations with (p_student_id, p_room_id, status = 'Pending'). Return success.

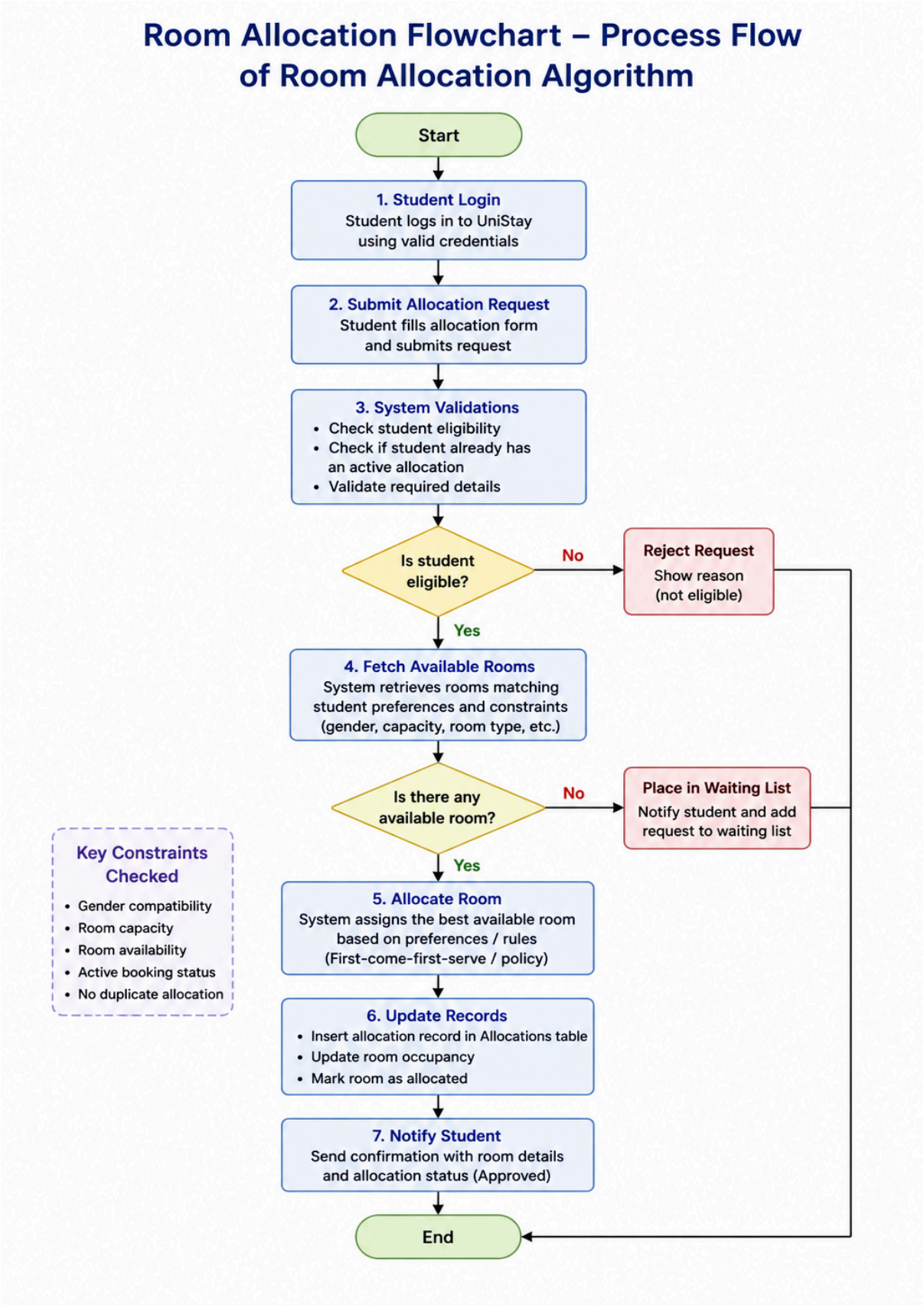

Figure 3. Room Allocation Flowchart – Process flow of room allocation algorithm

Algorithm 2: Swap Approval Workflow

Input: Swap Request ID (p_swap_id), Admin ID (p_admin_id)

Output: Both students' room_id values are swapped atomically; swap status is updated to 'Approved'. On failure, all changes are rolled back.

- **Step 1:** Student A submits a swap request targeting Student B. System inserts a record into Swap_Requests with status = 'Pending'. The trg_prevent_cross_gender_swap trigger automatically blocks the insert if A and B have different genders.
- **Step 2:** Student B receives a notification and provides consent by updating target_consent = TRUE.
- **Step 3:** Admin views pending swap requests on the dashboard. Admin calls the ApproveSwap stored procedure with the swap_id.
- **Step 4:** BEGIN TRANSACTION. Lock the Swap_Requests row for the given swap_id. Verify that status is still 'Pending'; if not, ROLLBACK and raise an error.
- **Step 5:** Retrieve and lock the current room_id for Student A (requester) from Allocations WHERE status = 'Approved'.
- **Step 6:** Retrieve and lock the current room_id for Student B (target) from Allocations WHERE status = 'Approved'. If either student lacks an approved allocation, ROLLBACK and raise an error.
- **Step 7:** UPDATE Student A's allocation: SET room_id = Student B's room_id. UPDATE Student B's allocation: SET room_id = Student A's room_id.
- **Step 8:** UPDATE Swap_Requests SET status = 'Approved', admin_id = p_admin_id. COMMIT.

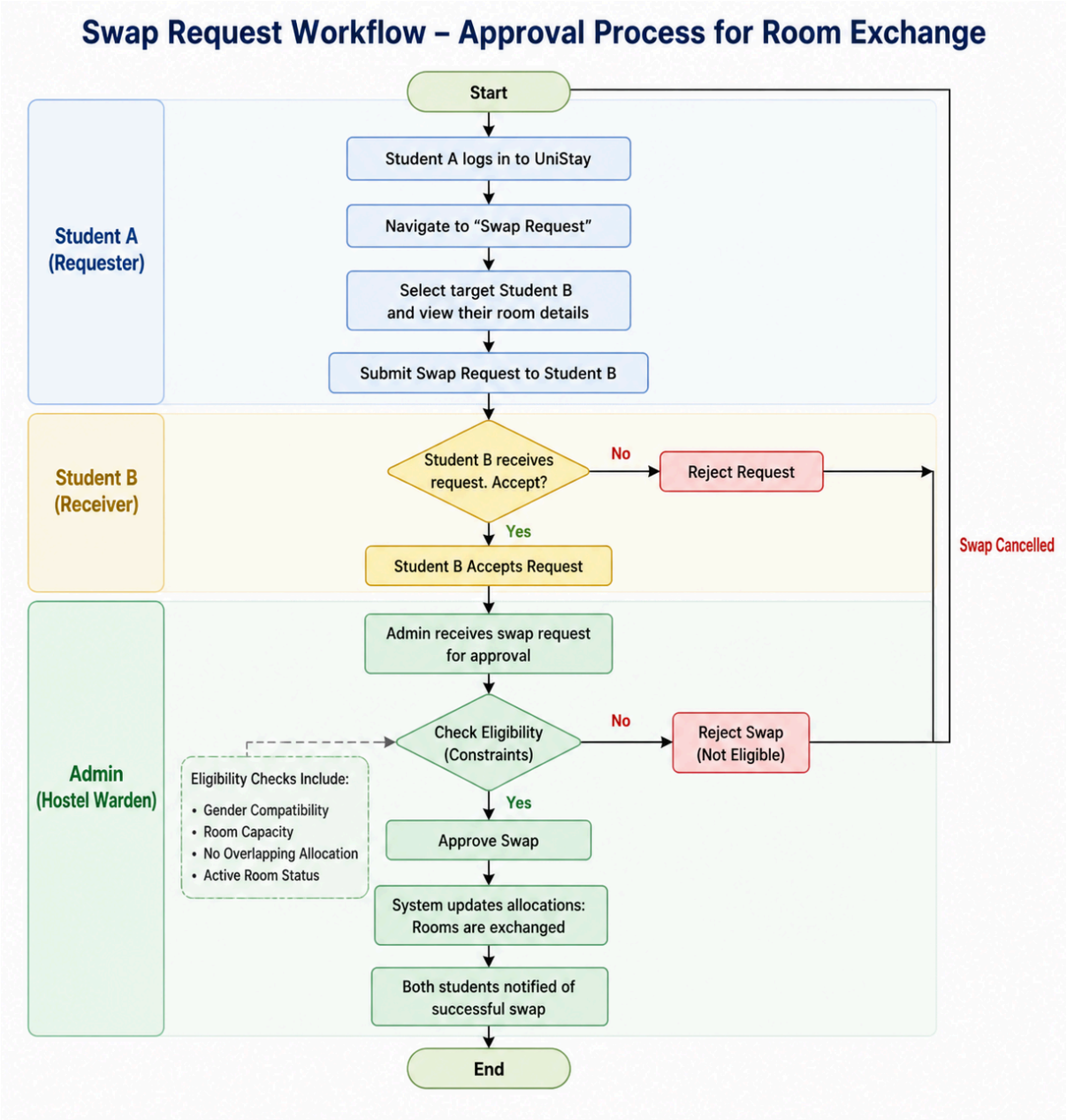

Figure 4. Swap Request Workflow – Approval process for room exchange

Discussion / Results

6.1 Working Scenarios

This section presents concrete scenarios that demonstrate the functioning of the UniStay system, illustrating how the database constraints, stored procedures, and triggers behave under real usage conditions.

Scenario 1: Successful Room Allocation

A male student (Aarav Sharma, student_id = 1) requests allocation to Room 101 in Ratan Tata Boys Hostel (room_id = 1, capacity = 2).

- The AllocateRoom procedure checks: no existing allocation for student 1 PASS.
- Gender check: student is Male, hostel is Boys PASS.
- Capacity check: 0 current approved occupants < 2 PASS.
- Outcome: INSERT into Allocations with status = 'Pending'. Admin later approves it.

Scenario 2: Blocked Allocation – Room Full

A third male student attempts to book Room 101 which already has 2 approved allocations (capacity = 2).

- Capacity check: 2 current approved occupants >= 2 FAIL.
- Outcome: Error raised 'Room is fully occupied. Allocation failed.' No record inserted.
- The trg_prevent_over_capacity trigger provides a second layer of protection at the database level, blocking even a direct INSERT if the room is at capacity.

Scenario 3: Blocked Allocation – Gender Mismatch

A female student attempts to book a room in Ratan Tata Boys Hostel.

- Gender check: student is Female, hostel is Boys, hostel is not Co-ed FAIL.
- Outcome: Error raised 'Gender mismatch: You cannot book a room in this hostel.' No record inserted.

Scenario 4: Successful Room Swap

Aarav Sharma (Room 101, Ratan Tata Boys) requests a swap with Rahul Yadav (Room 203, Bhagat Singh Boys). Both students are Male.

- trg_prevent_cross_gender_swap trigger: both students are Male PASS. Swap request inserted.
- Rahul provides consent. Admin calls ApproveSwap.
- Transaction begins. Both room_id values are exchanged atomically.

- Outcome: Aarav is now in Room 203; Rahul is now in Room 101. Swap status = 'Approved'.

Scenario 5: Blocked Swap – Cross-Gender

A male student attempts to create a swap request targeting a female student.

- trg_prevent_cross_gender_swap trigger fires on INSERT to Swap_Requests.
- Genders do not match FAIL.
- Outcome: Error raised 'Trigger Error: Cross-gender room swaps are strictly prohibited.' No swap request inserted.

6.2 Query Results

The following table shows the result of the View_Allocated_Students view, which lists all students with approved room allocations:

Student Name	Course	Hostel Name	Room Number	Allocation Date
Aarav Sharma	B.Tech CSE	Ratan Tata Boys Hostel	101	Current Date
Rahul Yadav	B.Tech ECE	Bhagat Singh Boys Hostel	203	Current Date
Kunal Mehta	Mech. Engg.	Vikram Sarabhai Boys Hostel	301	Current Date
Rohan Verma	B.Tech ECE	Bhagat Singh Boys Hostel	201	Current Date
Arjun Mishra	Mech. Engg.	Homi Bhabha Boys Hostel	401	Current Date
Riya Verma	B.Tech CSE	Kalpana Chawla Girls Hostel	102	Current Date
Priya Singh	B.Tech ECE	Gargi Girls Hostel	204	Current Date
Ananya Sharma	B.Tech CSE	Kalpana Chawla Girls Hostel	103	Current Date
Meera Iyer	B.Tech ECE	Gargi Girls Hostel	201	Current Date

Table 6: Query Result – Approved Allocations (from View_Allocated_Students)

The following table shows the result of the swap request scenario:

Swap ID	Requester	Target Student	Status	Admin Action
1	Aarav Sharma	Rahul Yadav	Pending	Awaiting admin review
2	Riya Verma	Ananya Sharma	Pending	Awaiting admin review

Table 7: Sample Swap Requests in the System

6.3 SQL Implementation

This section presents the key SQL queries used in UniStay, along with explanations of why each query is used and what it does. The majority of the SQL logic was designed and implemented by the team. Only minor assistance (syntax correction and optimization suggestions) was taken from AI tools.

DDL – Table Creation

Why: The CREATE TABLE statements define the entire data model of the system. They establish primary keys, foreign keys, CHECK constraints, and UNIQUE constraints that enforce data integrity at the database level.

What: The following statement creates the Allocations table, which tracks every room assignment in the system:

```
CREATE TABLE Allocations (
    allocation_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT UNIQUE NOT NULL,
    room_id INT NOT NULL,
    allocation_date DATE DEFAULT (CURRENT_DATE),
    status VARCHAR(20) DEFAULT 'Pending'
        CHECK (status IN ('Pending', 'Approved', 'Rejected')),
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
        ON DELETE CASCADE,
    FOREIGN KEY (room_id) REFERENCES Rooms(room_id)
        ON DELETE CASCADE
);
```

DML – INSERT (Seed Data)

Why: INSERT statements populate the database with initial data. In UniStay, they are used to add hostel records, student accounts, and demo allocations so that the system can be demonstrated with realistic data.

What: The following inserts sample students and their allocations:

```
INSERT INTO Students (name, gender, course, year,
    contact_number, email, password) VALUES
('Aarav Sharma', 'Male', 'B.Tech CSE', 2,
    '9876501001', 'aarav@student.edu', 'pwd1'),
('Riya Verma', 'Female', 'B.Tech CSE', 2,
    '9876501008', 'riya@student.edu', 'pwd8');

INSERT INTO Allocations (student_id, room_id, status) VALUES
(1, 1, 'Approved'),  -- Aarav Sharma -> Room 101
(8, 14, 'Approved'); -- Riya Verma   -> Room 102, Kalpana Chawla Girls
```

DML – UPDATE (Admin Approval)

Why: UPDATE is used by the admin to change an allocation's status from 'Pending' to 'Approved' or 'Rejected'. This is the core admin action in the allocation workflow.

What: The following query approves a specific student's pending allocation:

```
-- Admin approves Vikram Patel's pending allocation
UPDATE Allocations
SET status = 'Approved'
WHERE student_id = 7;
```

DML – DELETE

Why: DELETE allows the system to remove cancelled or erroneously created allocations. It is protected by FOREIGN KEY CASCADE constraints so that deleting a student automatically removes their allocation.

What: The following query demonstrates a manual deletion of a specific allocation:

```
-- Remove a specific pending allocation (e.g., student cancels request)
DELETE FROM Allocations
WHERE student_id = 13 AND room_id = 10;
```

DQL – JOIN Query

Why: JOIN queries are the primary way to retrieve meaningful data by combining information from multiple related tables. They eliminate the need to store redundant data.

What: This query retrieves the full allocation details — student name, room number, hostel name, and approval status — by joining four tables:

```
SELECT
    s.name AS student_name,
    r.room_number,
    h.hostel_name,
    a.status AS allocation_status
FROM Students s
JOIN Allocations a ON s.student_id = a.student_id
JOIN Rooms r ON a.room_id = r.room_id
JOIN Hostels h ON r.hostel_id = h.hostel_id
WHERE a.status = 'Approved';
```

DQL – GROUP BY + HAVING

Why: GROUP BY aggregates rows by a common attribute (hostel), and HAVING filters groups based on an aggregate condition. This is used to identify rooms or hostels that exceed a certain occupancy threshold.

What: This query finds all rooms with 2 or more approved occupants:

```
SELECT
    r.room_number,
    COUNT(a.student_id) AS current_occupancy
FROM Rooms r
JOIN Allocations a ON r.room_id = a.room_id
WHERE a.status = 'Approved'
GROUP BY r.room_id, r.room_number
HAVING COUNT(a.student_id) >= 2;
```

DQL – Subquery

Why: Subqueries allow the system to use the result of one query as the input to another. This is used to find students who have not yet been allocated a room.

What: This query lists all students who have no pending or approved allocation:

```
SELECT name, email, course
FROM Students
WHERE student_id NOT IN (
    SELECT student_id
    FROM Allocations
    WHERE status IN ('Approved', 'Pending')
);
```

Stored Procedure – AllocateRoom

Why: A stored procedure encapsulates multi-step business logic in the database itself. AllocateRoom ensures that all three pre-allocation checks (no duplicate, gender match, capacity available) are performed atomically before inserting a new record.

What: The procedure takes a student_id and room_id, runs three validation checks, and inserts a 'Pending' allocation if all checks pass. An error is raised using SIGNAL SQLSTATE '45000' if any check fails. The full procedure code is provided in the advanced.sql file.

Swap Logic – ApproveSwap Procedure

Why: A room swap involves updating two rows simultaneously. Using a transaction with BEGIN/COMMIT and FOR UPDATE row-level locks ensures that no race condition can produce an inconsistent state (e.g., both students ending up in the same room).

What: The procedure fetches both students' current room_ids, performs a cross-update, marks the swap as Approved, and commits. If any step fails, ROLLBACK reverts all changes. The full procedure code is in advanced.sql.

```
-- Core swap logic within the ApproveSwap procedure:
UPDATE Allocations
   SET room_id = v_tgt_room_id
   WHERE student_id = v_requester_id AND status = 'Approved';

UPDATE Allocations
   SET room_id = v_req_room_id
   WHERE student_id = v_target_id AND status = 'Approved';

UPDATE Swap_Requests
   SET status = 'Approved', admin_id = p_admin_id
   WHERE swap_id = p_swap_id;
```

Code: Backend – Database Connection (index.js)

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
  host:     process.env.DB_HOST || 'localhost',
  user:     process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_NAME  || 'hostel_db',
```

```
  port:     process.env.DB_PORT  || 3306,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

module.exports = {
  query: (text, params) => pool.execute(text, params),
  pool: pool
};
```

Code: Sample Frontend – Room Booking Form (HTML/CSS)

```
<!-- Student Room Booking Page -->
<!DOCTYPE html>
<html lang='en'>
<head>
  <meta charset='UTF-8'>
  <title>UniStay – Book a Room</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f6f8; }
    .container { max-width: 500px; margin: 60px auto; background: #fff;
                 padding: 32px; border-radius: 8px; box-shadow: 0 2px 8px #0001; }
    h2 { color: #2E75B6; }
    label { display: block; margin-top: 16px; font-weight: bold; }
    select, input { width: 100%; padding: 10px; margin-top: 6px;
                    border: 1px solid #ccc; border-radius: 4px; }
    button { margin-top: 24px; width: 100%; padding: 12px;
             background: #2E75B6; color: #fff; border: none;
             border-radius: 4px; font-size: 16px; cursor: pointer; }
  </style>
</head>
<body>
  <div class='container'>
    <h2>Book a Room</h2>
    <label>Select Room</label>
    <select id='roomSelect'></select>
    <button onclick='submitBooking()'>Submit Booking Request</button>
  </div>
  <script>
    async function submitBooking() {
      const roomId = document.getElementById('roomSelect').value;
```

```
      const res = await fetch('/api/allocate', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ room_id: roomId })
      });
      const data = await res.json();
      alert(data.message);
    }
  </script>
</body>
</html>
```

Concluding Remarks

UniStay successfully demonstrates how a well-designed relational database system can solve a real-world administrative challenge. The project delivered a fully functional Hostel Room Allocation and Swap Management System that automates the key workflows of hostel administration room allocation, approval, and student-to-student swaps within a structured, constraint-enforced digital environment.

The database design employs all major relational concepts studied in the DBMS course: *normalization to 3NF to eliminate redundancy, foreign key relationships to enforce referential integrity, CHECK constraints to validate domain rules, UNIQUE constraints to prevent duplicate allocations, stored procedures to encapsulate complex multi-step logic, database triggers to enforce rules at the lowest possible level, views to simplify application queries, and transactions with row-level locking to handle concurrent updates safely.*

The system was tested against multiple edge cases including attempts to over-allocate rooms, cross-gender allocations, duplicate allocation requests, and concurrent swap approvals and demonstrated robust constraint enforcement in all scenarios. The team successfully implemented the full stack: database schema, seed data, advanced SQL features, Node.js backend with a connection pool, and a basic HTML/CSS frontend.

This project reinforced the practical value of database-level constraint enforcement as a complement to application-level validation. By pushing critical business rules (capacity checks, gender compatibility, swap integrity) into the database through triggers and stored procedures, the system ensures correctness regardless of how the database is accessed. This makes UniStay a reliable foundation on which future enhancements can be safely built.

Societal Relevance

UniStay addresses a genuine and widespread challenge faced by educational institutions across India and globally. The benefits of the system extend beyond technical achievement to create meaningful social impact:

- Fairness and Transparency: The automated allocation system ensures that all students are subject to the same rules and constraints. There is no room for favoritism or manual bias; every request is processed by the same algorithmic logic. This promotes a sense of fairness among students.
- Accessibility and Inclusion: By enabling students to browse and request rooms online, the system removes barriers for students who cannot easily visit hostel offices in person including students with disabilities, students from remote areas arriving late, or students dealing with personal challenges.
- Reduced Administrative Burden: Wardens and hostel administrators spend significant time on manual record-keeping and in-person request processing. UniStay automates these tasks, freeing administrators to focus on student welfare, campus safety, and other high-value responsibilities.
- Environmental Benefit: Eliminating paper-based application forms, registers, and approval letters reduces paper consumption across the institution a small but meaningful contribution to environmental sustainability.
- Digital Literacy: Developing and deploying a system like UniStay contributes to the broader goal of digitizing campus administration in Indian universities, supporting the national vision of a Digital India and preparing institutions for further technological adoption.
- Student Well-being: Accommodation stress is a significant contributor to student anxiety, particularly for first-year students. UniStay's real-time availability tracking and transparent approval process reduce uncertainty and allow students to secure accommodation with confidence before the academic year begins.
- Scalability for Larger Institutions: The architecture of UniStay a normalized relational database with stored procedures and triggers is inherently scalable. The same system can be adapted for institutions with thousands of students and dozens of hostels, making it relevant beyond a single campus.

Future Scope

The current implementation of UniStay establishes a solid foundation. The following enhancements are planned or proposed for future development:

- Mobile Application: Development of native iOS and Android applications (or a Progressive Web App) that would allow students to book rooms, track request status, and receive push notifications for approval updates directly on their smartphones significantly improving accessibility.
- Smart Allocation Algorithm: Integration of an intelligent room recommendation engine that suggests rooms based on student preferences (e.g., preferred hostel type, roommate preferences, course batch), historical satisfaction data, and optimization criteria such as minimizing walk time to academic buildings.
- Online Fee Payment Integration: Adding a payment gateway to allow students to pay hostel fees directly through the platform, generating digital receipts and maintaining a payment audit trail in the database.
- Complaint and Maintenance Management: A module for students to raise maintenance requests (broken fixtures, plumbing issues, etc.) and for wardens to track and resolve them — extending the platform beyond allocation to full hostel lifecycle management.
- Automated Notifications: Integration with email (SMTP) and SMS gateways to automatically notify students and admins at each stage of the allocation and swap workflow — reducing the need for manual follow-ups.
- Analytics Dashboard for Administration: A reporting module for university administration providing hostel occupancy trends, peak demand periods, gender-wise distribution, and allocation efficiency metrics supporting data-driven decision making.
- Multi-Hostel / Multi-Campus Support: Extending the system to support multiple campuses or institutions, with role-based access control for hostel-specific wardens, campus-level admins, and university-wide administrators.
- API Integration with University ERP: Connecting UniStay with the university's existing ERP system to automatically verify student enrolment status, sync student records, and integrate with the academic calendar for annual re-allocation cycles.

Team Details and Contribution

The UniStay project was a collaborative effort by a team of four students. All four members contributed equally to every phase of the project, from initial design through final testing and documentation.

Member Name	Contribution Areas
Ananya Upadhyay	Database Design (ER Diagram, Schema), SQL Implementation (DDL), Report Documentation, System Testing, Presentation Preparation
Annu	Database Design (Normalization), SQL Implementation (Stored Procedures, Views), Frontend Development (HTML/CSS), Report Documentation, Testing.
Abhinav Sharma	SQL Implementation (DQL Queries, Seed Data), System Integration, Report Documentation, Presentation Preparation
Aditya Narayan Singh	Backend Development (Node.js, API Routes, DB Connection), SQL Implementation (Triggers), Report Documentation, Testing

Table 9: Team Details and Individual Contributions

Equal Contribution Statement

All four team members Ananya Upadhyay, Annu, Abhinav Sharma, and Aditya Narayan Singh contributed equally to the project. Each member was involved in all four key phases: Design, Implementation, Documentation, and Testing. Task division was done for efficiency, but all members reviewed and understood each other's work.

The SQL table creation and database structure were primarily designed and implemented by the team, drawing on concepts studied in the DBMS course. The stored procedures, triggers, views, and index strategies were developed by the team through collaborative effort and iterative refinement.

AI Tools Acknowledgement

We acknowledge that limited assistance was taken from AI tools during the development of this project. The AI tools used and the specific nature of their use are described below:

AI Tool	Purpose of Use
ChatGPT (OpenAI)	Understanding DBMS concepts; debugging minor SQL syntax errors; reviewing stored procedure logic structure

AI Tool	Purpose of Use
Claude (Anthropic)	Clarifying normalization concepts; reviewing trigger syntax; improving code readability and comments
Cursor (Anysphere)	Code completion suggestions within the IDE during Node.js backend development
Antigravity (Google)	Reviewing bcrypt password hashing implementation for the admin password storage approach, Frontend, Node.js logic and implementations.
Grok (xAI)	Quick reference queries about MySQL-specific syntax (DELIMITER, SIGNAL SQLSTATE)

Important: The core logic of the system — including the database schema design, ER diagram, normalization analysis, stored procedure and trigger business logic, query design, and overall system architecture — was conceived and implemented by the team. AI tools were used exclusively for:

- Understanding and clarifying concepts already introduced in the course
- Debugging small, isolated syntax errors
- Improving the readability and inline documentation of existing code

No AI tool was used to generate any complete section of code, design any database structure, or write any significant portion of this report. All implementation choices, and documentation were the original work of the team members.

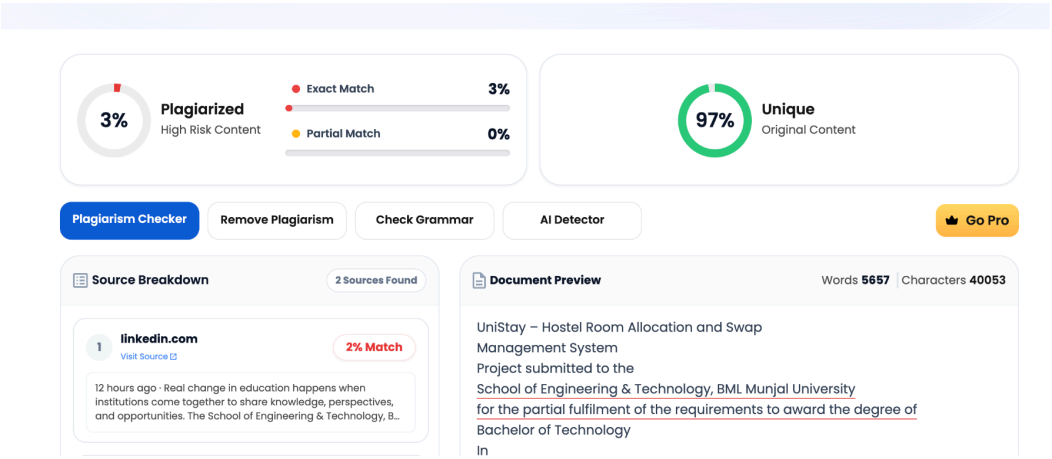

Figure 5. Plagiarism check

References

- Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, 7th Edition, Pearson Education, 2015.
- Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, Database System Concepts, 7th Edition, McGraw-Hill Education, 2019.
- MySQL 8.0 Reference Manual – Stored Procedures, Triggers, and Views. Oracle Corporation. Available at: https://dev.mysql.com/doc/refman/8.0/en/ [Accessed May 2026].
- Node.js Documentation – Official Node.js runtime documentation. Available at: https://nodejs.org/en/docs/ [Accessed May 2026].
- mysql2 npm Package – MySQL client for Node.js. Available at: https://www.npmjs.com/package/mysql2 [Accessed May 2026].
- MDN Web Docs – HTML and CSS Reference. Mozilla Developer Network. Available at: https://developer.mozilla.org/ [Accessed May 2026].
- OpenAI ChatGPT – AI assistant used for limited concept clarification and syntax debugging. Available at: https://chat.openai.com/ [Accessed during project development, 2025–2026].
- Anthropic Claude – AI assistant used for limited concept clarification and code review. Available at: https://claude.ai/ [Accessed during project development, 2025–2026].
- Cursor IDE – AI-powered code editor used for code completion during backend development. Available at: https://www.cursor.com/ [Accessed during project development, 2025–2026].
- Antigravity (Google AI Tool) – Used for understanding bcrypt hashing. Available at internal Google tooling [Accessed during project development, 2025–2026].
- Grok (xAI) – AI assistant used for MySQL-specific syntax reference. Available at: https://grok.x.ai/ [Accessed during project development, 2025–2026].
- BML Munjal University – School of Engineering & Technology, DBMS Course Materials, Academic Year 2025–2026.